

Relatório Técnico — TrafficAI

Sistema de Simulação e Gestão Inteligente de Tráfego Urbano

Licenciatura em Engenharia Informática

Projeto Final de Curso

Autor: Beatriz Antunes · Data: Maio 2026

Resumo

O **UrbanFlow** é uma plataforma web de simulação e análise de tráfego urbano desenvolvida como projeto final de curso de Licenciatura em Engenharia Informática. O sistema combina um motor de simulação em JavaScript, inteligência artificial adaptativa, aprendizagem por reforço (Q-Learning) e um motor de regras em linguagem natural para modelar e comparar estratégias de controlo semafórico em ambiente urbano.

A plataforma integra dados reais de transporte público português através do formato GTFS (18 distritos) e disponibiliza múltiplas vistas analíticas — métricas em tempo real, comparação AI vs. controlo tradicional, análise ambiental e de segurança, e histórico de simulações. Os resultados experimentais demonstram reduções consistentes de 40–55% no tempo médio de espera e de 25–35% nas emissões de CO₂ face ao controlo de ciclo fixo, validando a eficácia do controlo adaptativo em cenários urbanos.

A plataforma encontra-se publicamente implantada em ambiente cloud (frontend em Vercel, backend em Render), recorrendo exclusivamente a serviços gratuitos.

Palavras-chave: simulação de tráfego, controlo semafórico adaptativo, aprendizagem por reforço, processamento de linguagem natural, GTFS, emissões de CO₂, React, Python

Abstract

UrbanFlow is a web-based urban traffic simulation and analysis platform developed as a final-year Computer Engineering project. The system combines a JavaScript simulation engine, adaptive artificial intelligence, reinforcement learning (Q-Learning), and a natural language rule engine to model and compare traffic signal control strategies in an urban environment. The platform integrates real Portuguese public transport data via the GTFS format (18 districts) and provides multiple analytical views — real-time metrics, AI vs. traditional control comparison, environmental and safety analysis, and simulation history. Experimental results consistently show reductions of 40–55% in average vehicle wait time and 25–35% in CO₂ emissions compared to fixed-cycle control, validating the effectiveness of adaptive control in urban scenarios.

The platform is publicly deployed in a cloud environment (frontend on Vercel, backend on Render), using exclusively free-tier services.

Keywords: traffic simulation, adaptive traffic signal control, reinforcement learning, natural language processing, GTFS, CO₂ emissions, React, Python

Índice

Relatório Técnico — TrafficAI	1
Resumo	2
Abstract	2
1. Introdução	5
Objetivos	5
2. Estado da Arte.....	6
2.1 Ferramentas de Simulação de Tráfego Urbano.....	6
2.2 Controlo Semafórico com Inteligência Artificial	7
2.3 Aprendizagem por Reforço para Gestão de Semáforos	7
2.4 Processamento de Linguagem Natural para Tráfego.....	8
2.5 Integração de Dados GTFS.....	8
2.6 Posicionamento do UrbanFlow	9
3. Engenharia de Requisitos e Modelação	9
3.1 Levantamento de Requisitos.....	9
3.2. Modelação Funcional(UML)	13
7.2 Fluxo — Aplicar Regra de Tráfego (UC03)	17
7.3 Fluxo — Comparação com Shadow Engine (UC05).....	18
Arquitetura do Sistema	18
Fluxo de Dados da Simulação	19
Modelo de Dados.....	19
Objeto de Métricas	19
Regra Estruturada (output do Rule Engine)	20
Entrada no Histórico (localStorage)	20
Configuração de Interseção	20
3.4. Modelação de Dados (UML/DER).....	21
4. Setup do Ambiente e Ferramentas	22
5. Planeamento e Desenvolvimento Incremental	23
6.Discussão Crítica do Fluxo de Vibe Coding	24
3. Descrição Geral do Sistema.....	31
3.1 Visão Geral	31
3.2 Modos de Operação	31
3.3 Cenários Disponíveis.....	31
3.4 Tipos de Entidades Simuladas	31
4. Atores do Sistema	32
12. Diagramas de Sequência	32
12.1 Iniciar Simulação (UC02)	32

12.2 Aplicar Regra de Tráfego (UC03)	32
12.3 Comparação AI vs. Tradicional (UC05).....	33
12.4 Carregar Mapa GTFS (UC06)	33
12.5 Guardar Simulação no Histórico (UC10)	34
13. Diagrama de Atividades	34
13.1 Ciclo Principal de Simulação (tick)	34
13.2 Processamento de Regra em Linguagem Natural	35
14. Diagrama de Componentes	36
15. Mockups da Interface	37
15.1 Vista — Dashboard de Métricas (Overview)	37
15.2 Vista — Live Maps	37
15.3 Vista — GTFS Map	38
15.4 Vista — Analytics.....	39
15.5 Vista — Reports	39
16. Implementação.....	40
16.1 Motor de Simulação (SimulationEngine)	40
16.2 Controlo Semafórico — Modo AI	40
16.3 Controlo Semafórico — Modo RL (Q-Learning)	41
16.4 Motor de Regras em Linguagem Natural	41
16.5 Integração de Perfis GTFS	42
16.6 Geração Pseudo-aleatória Reprodutível	42
17. Testes e Validação	42
17.1 Estratégia de Testes	42
17.2 Cenários de Teste por Caso de Uso	43
17.3 Testes de Robustez	43
17.4 Correções Identificadas e Resolvidas	44
17.5 Implantação e Disponibilidade Pública.....	44
18. Conclusão e Trabalho Futuro	44
18.1 Sumário do Projeto.....	44
18.2 Limitações.....	45
18.3 Trabalho Futuro.....	45
19. Referências.....	46

1. Introdução

O **UrbanFlow** é uma plataforma web de simulação e análise de tráfego urbano que combina inteligência artificial, dados reais de transporte público (GTFS) e uma engine de regras em linguagem natural para modelar e comparar estratégias de controlo semafórico em ambiente urbano.

O sistema permite ao utilizador configurar cenários de tráfego, observar a simulação em tempo real, aplicar regras operacionais expressas em português corrente, e analisar o impacto dessas regras em métricas como emissões de CO₂, tempo de espera, fluxo de veículos e segurança rodoviária.

Objetivos

- Simular o comportamento de tráfego urbano sob diferentes cenários e modos de controlo
- Comparar quantitativamente o controlo por IA com o controlo tradicional (ciclo fixo)
- Integrar dados reais de redes de transporte público portuguesas (GTFS)
- Permitir a definição de regras operacionais em linguagem natural
- Registar e analisar o histórico de simulações

2. Estado da Arte

2.1 Ferramentas de Simulação de Tráfego Urbano

A simulação de tráfego é uma área de investigação madura, com várias ferramentas estabelecidas:

Ferramenta	Tipo	Modelo	Limitações face ao TrafficAI
SUMO (Simulation of Urban MObility)	Open-source, desktop	Microscópico	Complexidade elevada de configuração; sem interface web nativa; sem regras em linguagem natural
Aimsun	Comercial, desktop	Micro/meso/macro	Proprietário, custo elevado; sem integração GTFS direta; sem controlo adaptativo baseado em RL em tempo real
VISSIM (PTV)	Comercial, desktop	Microscópico	Proprietário; parametrização complexa; sem interface acessível a não especialistas
MATSim	Open-source, desktop	Agente-baseado	Foco em mobilidade diária de larga escala; não adequado para análise semafórica em tempo real
CARLA	Open-source, 3D	Micro + veículos autónomos	Focado em condução autónoma; sem análise de semáforos ou GTFS

Nenhuma destas ferramentas combina: interface web acessível, integração GTFS, motor de regras em linguagem natural e comparação em tempo real entre modos de controlo. O TrafficAI preenche este nicho para contextos educativos e de prototipagem rápida.

2.2 Controlo Semafórico com Inteligência Artificial

O controlo adaptativo de semáforos é um problema amplamente estudado. As abordagens mais relevantes incluem:

- **SCOOT** (Split Cycle Offset Optimisation Technique) e **SCATS** — sistemas adaptativos centralizados usados em cidades reais; requerem infraestrutura de sensores dedicada e não são acessíveis para simulação académica.
- **Diakaki et al. (2002)** — abordagem multivariável baseada em controlador regulador linear; demonstra ganhos de 10–20% em throughput mas requer calibração por cenário.
- **Webster (1958)** — fórmula clássica para duração ótima de ciclo semafórico em função do fluxo observado; ainda usada como referência em simuladores modernos.

O TrafficAI implementa uma heurística AI inspirada nestes trabalhos: timing dinâmico proporcional à diferença de filas e preempção de emergências, executada em tempo real no browser sem infraestrutura externa.

2.3 Aprendizagem por Reforço para Gestão de Semáforos

A aplicação de RL ao controlo de semáforos tem ganho tração significativa:

Trabalho	Algoritmo	Contribuição	Limitação
IntelliLight (Wei et al., 2018)	DQN com rede neuronal	Primeira aplicação de deep RL a semáforos urbanos com dados reais	Requer GPU; não reproduzível sem infraestrutura ML
PressLight (Wei et al., 2019)	Pressure-based RL	Função de recompensa baseada em “pressão” de fila; generaliza melhor	Multi-agente centralizado; complexidade de implementação elevada
MPLight (Chen et al., 2020)	RL com sharing de parâmetros	Coordenação eficiente de múltiplos semáforos	Dependente de simulador externo (CityFlow)

O TrafficAI usa Q-Learning tabulado com discretização de filas (5 buckets) e função de recompensa inspirada em PressLight, mas sem redes neurais — uma troca consciente entre capacidade e interpretabilidade, adequada ao contexto educativo e à execução no browser.

2.4 Processamento de Linguagem Natural para Tráfego

A interface em linguagem natural para configuração de sistemas de tráfego é uma área emergente com poucos trabalhos diretos. Estudos existentes focam-se maioritariamente em:

- **Chatbots de informação de transporte público** (ex.: “qual é o próximo autocarro para Lisboa?”) — não aplicável a controlo operacional.
- **LLMs para geração de configurações de simulação** — abordagem recente (2023–2024) mas dependente de APIs externas e com latência elevada para uso em tempo real.

O motor de regras do TrafficAI adota uma abordagem diferente: extração por tokens e expressões regulares, sem dependência de LLM, o que garante funcionamento offline e latência nula. Esta é uma decisão pragmática adequada ao âmbito do projeto, com a limitação de não suportar linguagem arbitrária.

2.5 Integração de Dados GTFS

O GTFS (General Transit Feed Specification) é o formato padrão de dados de transporte público, mantido pela Google desde 2006 e adotado pela maioria dos operadores europeus. Ferramentas de referência que o utilizam:

- **OpenTripPlanner** — planeamento de rotas multimodal; não inclui simulação de tráfego motorizado.
- **TransitLand** — agregador de feeds GTFS a nível global; focado em visualização, não em simulação.
- **Conveyal Analysis** — análise de acessibilidade com dados GTFS; sem simulação de veículos individuais.

O TrafficAI é, tanto quanto se conhece, uma das poucas plataformas de simulação de tráfego que usa perfis horários GTFS para calibrar dinamicamente o spawn de veículos, tornando os cenários portugueses geograficamente e temporalmente representativos.

2.6 Posicionamento do UrbanFlow

Face ao estado da arte, o TrafficAI posiciona-se como uma plataforma educativa e de prototipagem que:

- 1) **Combina** simulação em tempo real, RL, AI heurística e regras em linguagem natural numa única interface web acessível.
- 2) **Integra** dados reais portugueses (GTFS de 18 distritos) sem requerer configuração especializada.
- 3) **Compara** quantitativamente modos de controlo em paralelo (shadow engine), algo ausente nas ferramentas comerciais de uso geral.
- 4) **Funciona offline** no browser, sem dependência de GPU, servidor de ML ou infraestrutura de sensores.

A principal limitação face às ferramentas profissionais é a simplificação do modelo de tráfego (grelha cartesiana, sem geometria real de estradas), aceitável no contexto educativo e de demonstração.

3. Engenharia de Requisitos e Modelação

3.1 Levantamento de Requisitos

Requisitos Funcionais

RF01 — Configuração da Simulação

O sistema deve permitir ao utilizador seleccionar: - Cenário de tráfego (normal, hora de ponta, acidente, emergência, por sistema) – Modo de sistema semafórico (AI, Tradicional, RL) – Velocidade de simulação (multiplicador de tempo) – Dimensão da grelha (número de interseções, de 2×2 a 10×10)

RF02 — Execução da Simulação em Tempo Real

O sistema deve: - Renderizar a grelha de interseções e veículos em canvas a uma cadência de ~300 ms – Fazer spawn de veículos e peões com base nos pesos do cenário ativo – Aplicar lógica semafórica (modo AI) ou de ciclo fixo (modo Tradicional) – Detetar e registar colisões e evitamentos de colisão

RF03 — Motor de Regras em Linguagem Natural

O sistema deve permitir ao utilizador escrever regras em português (ex.: “Priorizar ambulâncias na hora de ponta”) e: - Enviar o texto ao backend para interpretação – Em caso de falha do backend, executar a regra localmente com o fallback JS – Aplicar a regra estruturada resultante ao motor de simulação – Listar e remover regras ativas

RF04 — Dashboard de Métricas em Tempo Real

O 10imililar deve apresentar, em atualização 10imililar10: - Tempo médio de espera (veículos ligeiros e pesados separadamente) – Taxa de fluxo de veículos – Emissões totais de CO₂ (em kg, com base nos fatores EURO 6/EEA) – Tempo de resposta de emergência – Número de colisões e colisões evitadas – Veículos ativos e completados

RF05 — Comparação AI vs. Tradicional (Shadow Engine)

O 10imililar deve executar em paralelo uma shadow engine com o modo oposto ao selecionado, usando a mesma semente aleatória, e apresentar uma vista 10imililar10ve10 com: - Gráficos de barras e radar por métrica – Diferença percentual por indicador – Classificação de CO₂ (Bom / Aceitável / Elevado).

RF06 — Mapa GTFS Interativo

O 10imililar deve: - Carregar dados GTFS dos 18 distritos portugueses (paragens, rotas, formas) – Apresentar as paragens e linhas sobre mapa interativo (Leaflet) – Permitir filtrar por operador e tipo de rota – Permitir iniciar uma simulação diretamente a partir de um 10imililar10 selecionado no mapa

RF07 — Vista Ambiental

O 10imililar deve apresentar: - CO₂ acumulado com classificação semântica – Evolução temporal das emissões (gráfico de área) – Métricas de eficiência energética derivadas do modo de 10imililar

RF08 — Vista de Segurança

O 10imililar deve: - Calcular um índice de risco 10imililar com base em colisões, tempo de emergência e espera de peões – Apresentar alertas categorizados por severidade (alto, médio, baixo, ok) – Detalhar métricas por tipo de veículo (ligeiro, pesado, emergência)

RF09 — Relatórios Exportáveis

O 10imililar deve gerar relatórios com: - Resumo de todas as métricas da simulação 10imililar10 – Tendência de cada indicador (melhoria/degradação ao longo da sessão) – Comparação entre o modo ativo e o modo oposto – Opção de exportação (download)

RF10 — Histórico de Simulações

O 11imilar deve guardar automaticamente cada simulação ao pausar, persistindo em localStorage: - Data/hora, modo, cenário, CO₂ total, CO₂ AI, veículos completados, colisões – Lista de histórico com tendência de CO₂ entre simulações consecutivas – Estatísticas agregadas (total de simulações, melhor CO₂, 11imil) – Funcionalidade de limpar o histórico

RF11 — Persistência de Preferências de UI

O 11imilar deve persistir e restaurar automaticamente: - Estado e largura dos painéis laterais e da sidebar – Preferências de layout entre sessões (via localStorage)

RF12 — Integração com Perfil de Tráfego GTFS

Para cenários portugueses, o 11imilar deve ajustar dinamicamente o 11imilar11 de spawn e a composição da frota com base nos perfis horários de partidas reais extraídos dos feeds GTFS.

RF13 — Logs do Sistema

O 11imilar deve gerar e apresentar um registo cronológico de eventos da simulação em curso, incluindo: - Estado da sessão (cenário e modo ativos) – Taxa de fluxo atual – Eventos de colisão ou ausência de colisões – Tempo de resposta de emergência com classificação (ok / aviso) – Variação do tempo médio de espera entre janelas de amostragem – Opção de 11imilar11 os logs em formato CSV

Requisitos Não Funcionais

RNF01 — Desempenho

- A simulação deve manter uma cadência de atualização de pelo menos 10 ticks/11imilar a velocidade ×1
- A interface deve ser 11imilar11ve durante a simulação sem bloquear o fio principal do browser
- A carga inicial dos dados GTFS deve ser assíncrona e não bloquear a UI

RNF02 — Disponibilidade e Resiliência

- O 11imilar deve funcionar em modo offline (sem backend): a simulação e o processamento de regras devem operar com os fallbacks JS integrados
- A ausência de dados GTFS não deve 11imilar o funcionamento da simulação (fallback para cenários genéricos)
- Erros de localStorage (quota excedida, indisponível) devem ser silenciados sem crash.

RNF03 — Usabilidade

- A interface deve ser operável sem manual, com controlos descritivos (tooltips, labels)
- Os painéis devem ser redimensionáveis por drag com feedback visual imediato
- As métricas críticas (CO₂, tempo de espera, colisões) devem ter classificação semântica por cor

RNF04 — Reprodutibilidade

- Duas simulações com a mesma semente, cenário e modo devem produzir resultados idênticos
- A semente é partilhada entre a engine principal e a shadow engine para garantir comparações justas

RNF05 — Precisão das Métricas Ambientais

- Os fatores de emissão de CO₂ devem ser baseados em dados reais (norma EURO 6 / EEA)
- As classificações de CO₂ devem usar limiares representativos de tráfego urbano real (≤50 kg: Bom; ≤200 kg: Aceitável; >200 kg: Elevado)

RNF06 — Segurança

- O input de regras em linguagem natural deve ser tratado como texto simples; nenhum input do utilizador deve ser executado como 12imila
- Não são armazenados dados pessoais ou sensíveis além das preferências de UI

RNF07 — Manutenibilidade

- A engine de simulação deve ser um 12imila JS puro sem dependências React, facilmente testável isoladamente
- O backend de regras deve ser uma função pura sem estado (sem base de dados)
- O fallback JS deve 12imilar fielmente a lógica Python do `rule_engine.py`

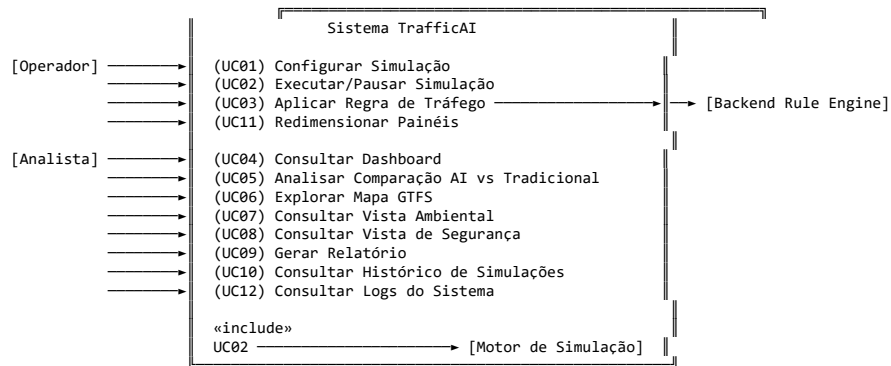
RNF08 — Portabilidade

- O frontend deve funcionar nos browsers modernos (Chrome, Firefox, Edge, Safari)
- O backend Python deve ser executável em ambiente Linux e Windows.

RNF09 — Escalabilidade

- A dimensão da grelha deve ser configurável (2–10), permitindo 13milhar desde 4 a 100 interseções
- O número máximo de veículos e peões é calculado dinamicamente em função do tamanho da grelha

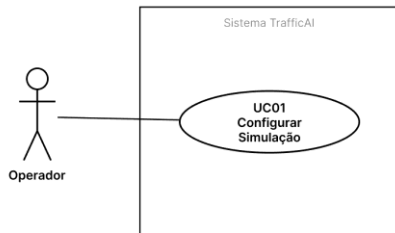
3.2. Modelação Funcional(UML)



Casos de Uso

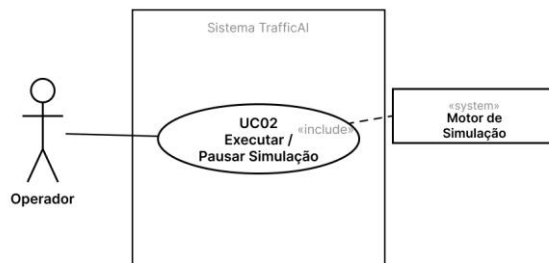
UC01 — Configurar Simulação

UC01 — Configurar Simulação



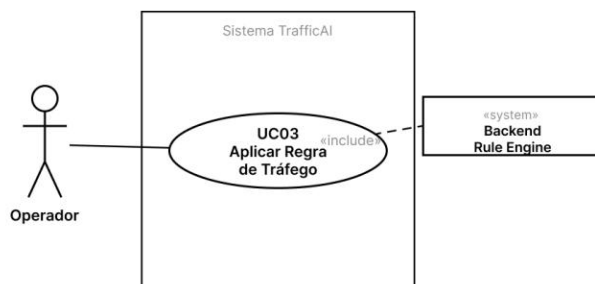
UC02 — Executar Simulação

UC02 — Executar / Pausar Simulação



UC03 — Aplicar Regra de Tráfego

UC03 — Aplicar Regra de Tráfego

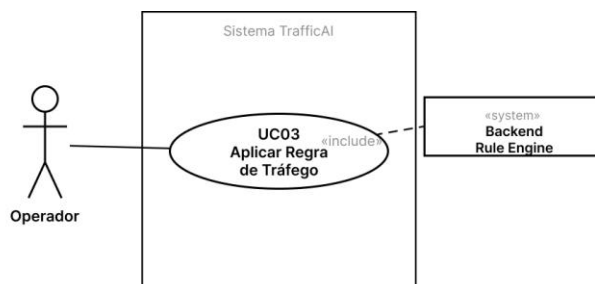


Exemplos de regras suportadas:

Entrada (português)	Tipo inferido	Alvo	Ação resultante
“Priorizar ambulâncias”	priority	ambulance	green_priority: true
“Extender verde 20 segundos para autocarros”	timing	bus	extend_green_seconds : 20
“Bloquear intersecção central”	block	all	block_intersection: true
“Descongestionar fila de carros”	density	car	reduce_wait_factor: 0.85
“Reduzir espera 30%”	density	all	reduce_wait_factor: 0.70

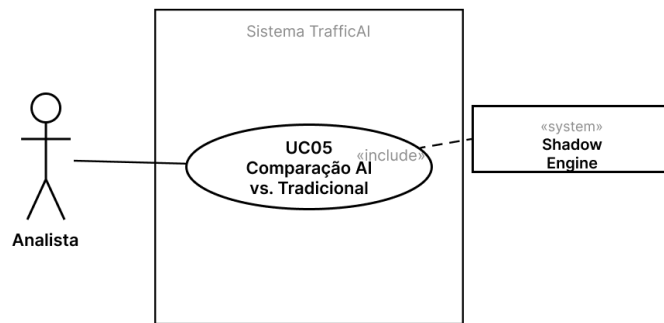
UC04 — Consultar Dashboard

UC03 — Aplicar Regra de Tráfego



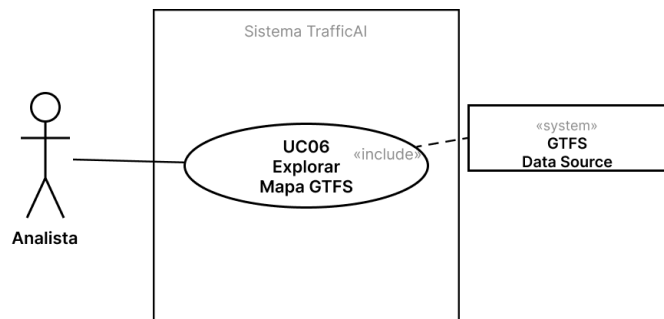
UC05 — Analisar Comparação AI vs. Tradicional

UC05 — Comparação AI vs. Tradicional



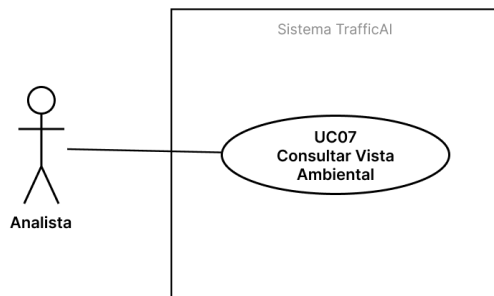
UC06 — Explorar Mapa GTFS

UC06 — Explorar Mapa GTFS



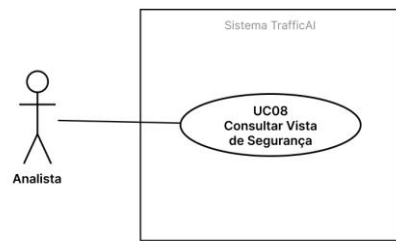
UC07 — Consultar Vista Ambiental

UC07 — Consultar Vista Ambiental



UC08 — Consultar Vista de Segurança

UC08 — Consultar Vista de Segurança

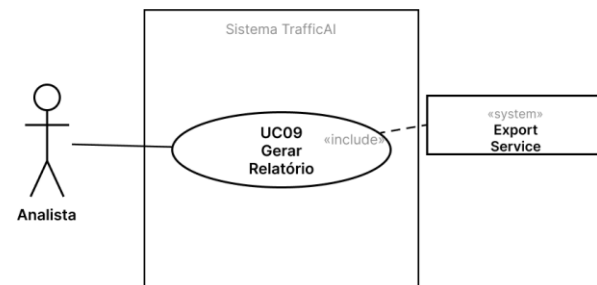


Limiares de alertas:

Condição	Severidade
<code>totalCollisions > 0</code>	Alto
<code>emergencyResponseTime > 18 s</code>	Médio
<code>pedestrianAvgWaitTime > 12 s</code>	Médio
<code>collisionAvoided ≥ 1</code>	Baixo (informativo)
Sem anomalias	OK

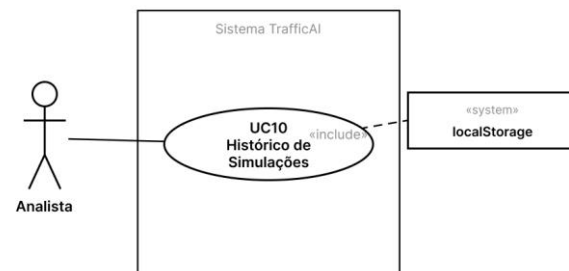
UC09 — Gerar Relatório

UC09 — Gerar Relatório



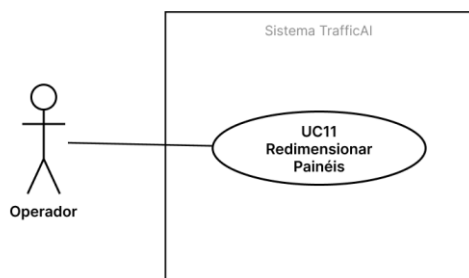
UC10 — Consultar Histórico de Simulações

UC10 — Histórico de Simulações



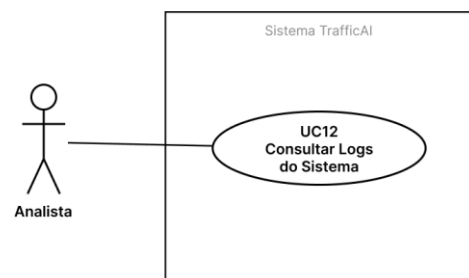
UC11 — Redimensionar Painéis

UC11 — Redimensionar Painéis



UC12 — Consultar Logs do Sistema

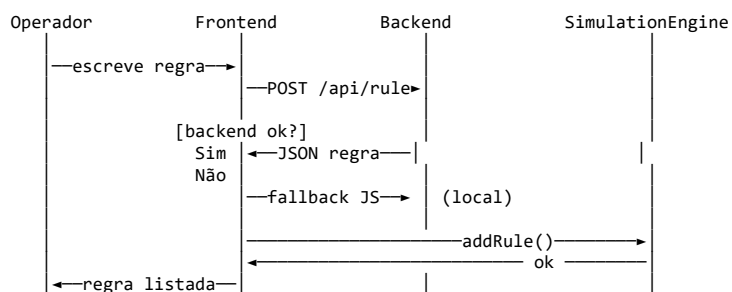
UC12 — Consultar Logs do Sistema



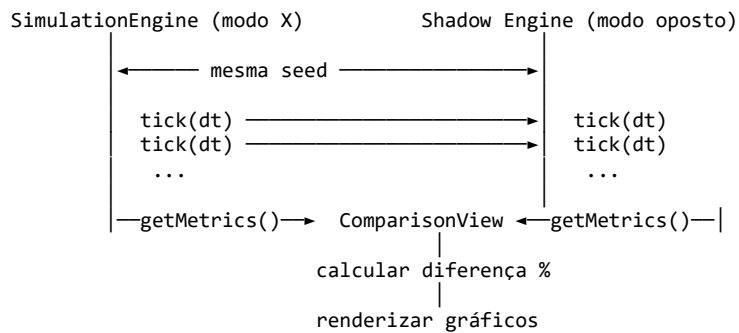
Categorias de log:

Categoria	Condição de aviso
session	Informação (sempre ok)
flow	Informação (sempre ok)
safety	Aviso se totalCollisions > 0
emergency	Aviso se emergencyResponseTime > 18 s
optimization	Aviso se variação de wait entre amostras > 2 s

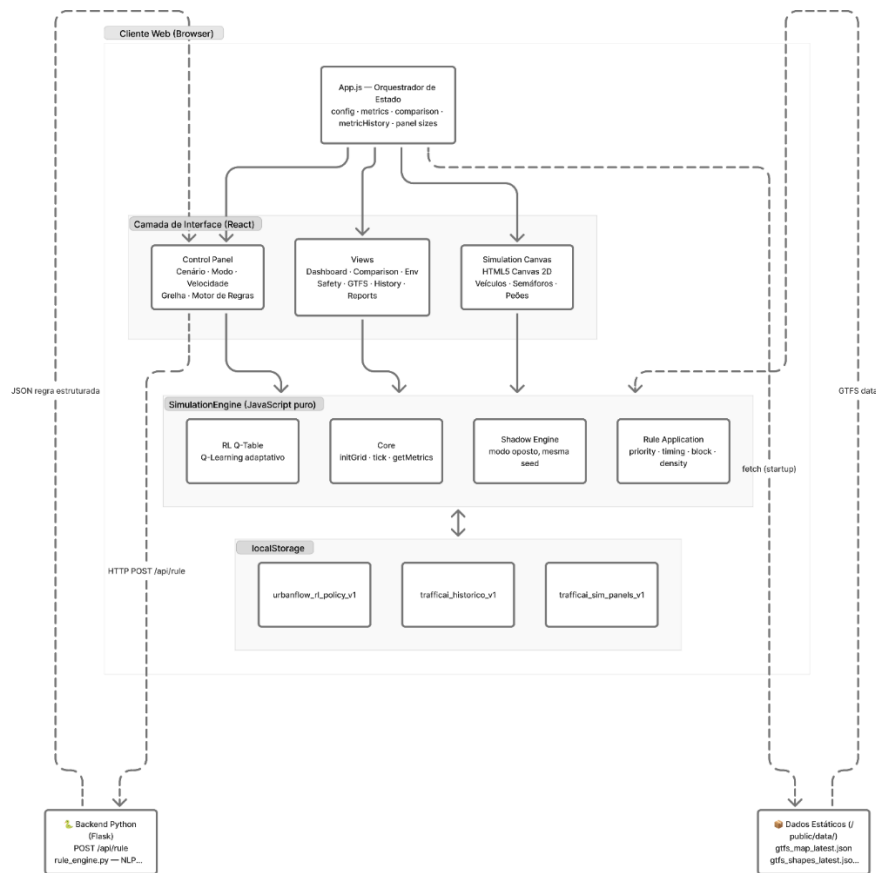
7.2 Fluxo — Aplicar Regra de Tráfego (UC03)



7.3 Fluxo — Comparação com Shadow Engine (UC05)

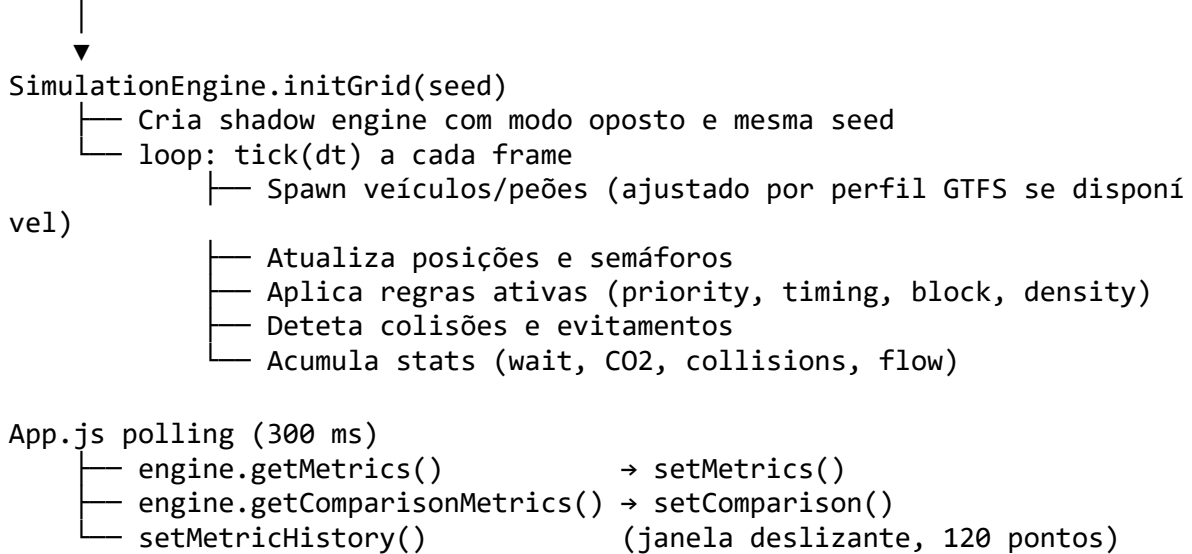


Arquitetura do Sistema



Fluxo de Dados da Simulação

config (cenário, modo, velocidade, grelha)



Modelo de Dados

Objeto de Métricas

```
{
  "avgWaitTime": 4.2,
  "flowRate": 12.5,
  "co2Emissions": 87.3,
  "emergencyResponseTime": 8.1,
  "totalCollisions": 2,
  "collisionAvoided": 5,
  "lightAvgWaitTime": 3.8,
  "heavyAvgWaitTime": 6.1,
  "lightVehiclesCompleted": 142,
  "heavyVehiclesCompleted": 23,
  "vehiclesActive": 18,
  "vehiclesCompleted": 165,
  "simulationTime": 312.4,
  "mode": "ai"
}
```

Regra Estruturada (output do Rule Engine)

```
{
  "type": "priority",
  "target": "ambulance",
  "conditions": {
    "time_start": null,
    "time_end": null,
    "scenario": "emergency",
    "location": "all"
  },
  "action": {
    "green_priority": true,
    "extend_green_seconds": null,
    "block_intersection": false,
    "reduce_wait_factor": null
  },
  "description_pt": "Priorizar ambulâncias em emergência"
}
```

Entrada no Histórico (localStorage)

```
{
  "id": "sim_1715000000000",
  "data": "2026-05-13T14:30:00.000Z",
  "modo": "ai",
  "cenario": "portugal_litoral",
  "co2": 134.7,
  "co2_ai": 98.2,
  "co2_tradicional": 171.3,
  "veiculos_completados": 312,
  "colisoas": 1,
  "tempo_medio_espera": 4.6
}
```

Configuração de Interseção

```
{
  "row": 2,
  "col": 1,
  "phase": 0,
  "timer": 0,
  "greenDuration": 20,
  "redDuration": 20,
  "blocked": false,
  "priorityTarget": null
}
```

3.4. Modelação de Dados (UML/DER)

O diagrama abaixo representa as principais entidades do sistema e as suas relações. As classes do lado do frontend (JavaScript) e do backend (Python) são apresentadas em conjunto, marcadas com o prefixo [JS] e [PY] respetivamente.



4. Setup do Ambiente e Ferramentas

O desenvolvimento do UrbanFlow assentou em quatro dimensões operacionais complementares, todas integradas com a metodologia de Vibe Coding descrita no capítulo 5. A escolha das ferramentas privilegiou **gratuidade, integração com Git e compatibilidade com fluxos assistidos por IA**, alinhando o ambiente operacional com a abordagem incremental adotada.

Dimensão Operacional	Ferramentas Adotadas	Descrição do Papel no Projeto
Documentação e Conhecimento	Notion como workspace central + Markdown estruturado (RELATORIO.md, README.md) versionado em Git	Repositório central de requisitos, modelação UML, decisões técnicas e atas de reunião. O Notion funcionou como "segundo cérebro" do projeto, agregando atas, backlog Kanban, rascunhos de modelação e notas de investigação; o Markdown versionado em Git suportou a documentação técnica formal (README, relatório). Esta separação refletiu a distinção entre artefactos exploratórios (Notion) e estáveis (Git).
Desenvolvimento e Vibe Coding	Modelos de IA generativa (Claude, Anthropic) como par programador, integrados com o editor local	Orquestração de código a partir de prompts estruturadas derivadas da modelação rigorosa do capítulo 3. A IA gerou o esqueleto do SimulationEngine, scaffolding dos componentes React, boilerplate do backend FastAPI e variantes algorítmicas para o módulo RL. A revisão crítica, decisões de arquitetura e integração final foram sempre humanas.
Controlo de Versões	Git + GitHub	Histórico de commits estruturado e descritivo, com mensagens semânticas. Cada funcionalidade foi desenvolvida em branches dedicadas (ex.: integração do módulo RL, fallback JS do rule engine), permitindo isolar experiências sem comprometer o ramo principal.
Gestão de Tarefas Agile	Quadros Kanban no Notion, com sprints semanais	Organização das tarefas em colunas "A Fazer / Em Curso / Bloqueado / Concluído", alinhadas com os marcos do Plano de Trabalho. Permitiu visibilidade contínua do progresso, priorização incremental e validação periódica com a equipa de orientação.
Implantação Contínua (CI/CD)	Vercel (frontend React) e Render (backend Python FastAPI), ambos em plano gratuito	Integração automática com o repositório Git: cada push para a branch principal desencadeia novo deploy sem intervenção manual. O frontend é servido como build estático otimizado pela Vercel; o backend corre num container Docker no Render. Permite manter a

		plataforma em produção sem custos de infraestrutura.
--	--	--

5. Planeamento e Desenvolvimento Incremental

O desenvolvimento do TrafficAI seguiu uma abordagem puramente iterativa (Agile) dividida em cinco fases cronológicas, executadas entre fevereiro e junho de 2026 (aproximadamente 16 semanas), em conformidade com o Plano de Trabalho aprovado pela equipa de orientação.

Fase Crítica	Horizonte Temporal	Descrição das Atividades e Metas
Fase 1: Fundação e Setup	Semanas 1–3 (2ª quinzena fevereiro – 1ª quinzena março)	Entrega do Plano de Trabalho e primeira reunião com a equipa de orientação. Estruturação do workspace de documentação e versionamento. Conclusão da revisão de literatura sobre simulação de tráfego, controlo semafórico adaptativo, Q-Learning e GTFS. Desenho inicial dos diagramas UML (casos de uso, classes, componentes) e modelação da estrutura base do ambiente de simulação com agentes parametrizáveis.
Fase 2: Prototipagem com Vibe Coding	Semanas 4–6 (2ª quinzena março – 1ª quinzena abril)	Conversão direta dos diagramas UML em prompts estruturadas para a ferramenta de IA generativa. Geração do esqueleto da aplicação React, do backend FastAPI e dos wireframes/interfaces iniciais. Desenvolvimento do motor de regras em linguagem natural, traduzindo instruções em português corrente (ex.: "Priorizar ambulâncias") em lógica executável. Conclusão com a apresentação intermédia (Pitch) a 7 de abril.
Fase 3: Implementação da Core Logic	Semanas 7–10 (2ª quinzena abril – 1ª quinzena maio)	Integração de algoritmos de Aprendizagem por Reforço (Q-Learning tabulado) para otimização autónoma das políticas semafóricas: definição da função de recompensa, discretização do espaço de estados e persistência da Q-Table. Desenvolvimento completo da interface gráfica de visualização (canvas de simulação, dashboards, vistas analíticas) e integração dos perfis horários GTFS dos 18 distritos portugueses. Acoplamento entre UI e lógica de negócio.
Fase 4: Testes, Refatorização e Validação	Semanas 11–13 (2ª quinzena maio – 1ª quinzena junho)	Desenvolvimento de testes funcionais por caso de uso, assistidos por IA na geração de cenários e asserções. Validação de invariantes do motor (reprodutibilidade por seed, conservação de métricas). Execução de testes em cenários extremos (Hora de Ponta, Acidente, Emergência) com comparação quantitativa entre modos AI, Tradicional e RL via shadow engine. Refactoring para otimização de performance e eliminação de bugs.
Fase 5: Deploy e Redação Final	Semanas 14–16 (1ª – 2ª quinzena junho)	Deploy estável em ambiente cloud com recurso exclusivo a serviços gratuitos: frontend em Vercel e backend em Render, ambos com integração contínua via Git. Transposição da documentação técnica para o formato final do Relatório de

		Execução, desenho do Poster Científico (entrega até 12 de junho) e preparação da defesa final perante o júri.
--	--	---

6.Planeamento e Desenvolvimento Incremental

Este capítulo apresenta os entregáveis produzidos ao longo do projeto, em conformidade com os requisitos da Unidade Curricular, e discute criticamente a metodologia de Vibe Coding adotada. Cada subsecção descreve um entregável tangível e a sua relevância para a avaliação final.

6.1 Software Funcional

O UrbanFlow encontra-se implantado em ambiente cloud e publicamente acessível, recorrendo exclusivamente a serviços de plano gratuito. A integração com o repositório Git assegura implantação contínua: cada push para a branch principal desencadeia automaticamente um novo deploy.

Componente	Provedor	Plano	URL Público
Frontend (React)	Vercel	Hobby (gratuito)	[inserir URL Vercel]
Backend (Python/FastAPI)	Render	Free (gratuito)	[inserir URL Render]

Principais características do protótipo operacional:

- Simulação multiagente em tempo real com até 100 interseções e ~150 veículos simultâneos, mantendo cadência de 300 ms.
- Três modos de controlo semafórico (Tradicional, IA Heurística, Q-Learning) com comparação paralela via shadow engine.
- Motor de regras em linguagem natural portuguesa, com fallback JavaScript para funcionamento offline.
- Integração de perfis horários GTFS reais de 18 distritos portugueses.
- Sete vistas analíticas: Dashboard, Live Maps, GTFS Map, Comparação, Ambiental, Segurança, Histórico e Logs.
- Persistência local de histórico e preferências de UI (localStorage), sem necessidade de conta de utilizador.

O frontend é servido como build estático otimizado pela Vercel; o backend corre num container Docker no plano gratuito do Render, com cold-start aceitável (~30 s) dada a sua natureza opcional — o frontend possui fallback JavaScript equivalente para todas as operações do motor de regras, garantindo funcionamento ininterrupto mesmo quando o backend está inativo.

[Inserir aqui screenshot da página inicial da aplicação em produção.]

6.2 Repositório de Código

O código-fonte é mantido em repositório Git privado alojado no GitHub, com histórico de commits estruturado e mensagens semânticas. O repositório está dividido em duas pastas principais: frontend/ (aplicação React) e backend/ (serviço FastAPI).

Métrica do Repositório	Valor
Plataforma	GitHub (repositório privado)
URL do repositório	[inserir URL]
Linguagens principais	JavaScript (~65%), Python (~30%), CSS (~5%)
Branch principal	main
Branches de desenvolvimento	[inserir número] (ex.: feature/rl-qlearning, feature/gtfs-integration, fix/rule-engine-fallback)
Total de commits	[inserir número]
Período de atividade	Fevereiro 2026 – Junho 2026
Integração CI/CD	Webhooks automáticos para Vercel (frontend) e Render (backend)

A estrutura de commits segue convenções semânticas (feat:, fix:, refactor:, docs:, test:), facilitando a leitura do histórico e a rastreabilidade entre cada alteração e a respetiva motivação. O desenvolvimento de funcionalidades de risco — nomeadamente a integração do módulo RL e a refatorização do motor de regras — foi isolado em branches dedicadas, permitindo iteração sem comprometer o ramo principal.

[Inserir aqui screenshot do histórico de commits do GitHub, evidenciando a regularidade do desenvolvimento e a qualidade das mensagens. Sugere-se também um screenshot do gráfico de contribuições (Insights > Contributors) ou do gráfico de network/branches.]

6.3 Base de Conhecimento e Workspace

A documentação, planeamento e gestão do projeto foram centralizados num workspace Notion, utilizado como "segundo cérebro" do projeto. Toda a informação não-código — atas de reunião, requisitos em refinamento, decisões técnicas, referências bibliográficas, rascunhos de modelação e planeamento de sprints — foi registada e versionada neste espaço.

Componente do Workspace	Conteúdo
Atas de Reunião	Registo de cada reunião com a equipa de orientação (Prof. Frederico Branco, Prof. Emanuel Peres, Prof. João Paulo Moura), com decisões e ações de seguimento.
Backlog e Sprints	Quadro Kanban com tarefas distribuídas por colunas "A Fazer / Em Curso / Bloqueado / Concluído", organizadas em sprints semanais alinhados com os marcos do Plano de Trabalho.
Modelação UML (rascunhos)	Versões iniciais dos diagramas de casos de uso, classes, atividades e sequência antes da transposição para o relatório final.
Requisitos e Decisões Técnicas	Documento vivo de RF/RNF, com histórico de alterações e justificações. Banco de decisões de arquitetura (ex.: Q-Learning tabulado vs. deep RL, regex vs. LLM no rule engine).
Referências e Notas de Investigação	Compilação de papers, documentação técnica e benchmarking de soluções existentes (capítulo 2).
Prompts e Logs de Vibe Coding	Registo das prompts mais relevantes usadas na geração assistida de código, com notas sobre o que funcionou e o que falhou.

A separação entre o repositório Git (código + documentação técnica formal) e o workspace Notion (planeamento, notas, exploração) refletiu a distinção clássica entre artefactos estáveis e exploratórios. O Notion alimentou continuamente o contexto das prompts de Vibe Coding: ao iniciar cada sessão de desenvolvimento, o estado atual do projeto e as decisões já tomadas estavam consolidados num único local, reduzindo a fricção e os erros típicos de perda de contexto.

[Inserir aqui screenshot da página principal do workspace Notion, mostrando a estrutura geral.]

6.4 Discussão Crítica do Fluxo de Vibe Coding

A utilização de ferramentas de IA generativa como par programador no desenvolvimento do UrbanFlow revelou-se transformadora, mas exigiu disciplina metodológica para extrair valor sem comprometer a qualidade técnica.

Benefícios Observados

- Aceleração da prototipagem. A geração inicial do SimulationEngine, do backend FastAPI e dos componentes React de visualização foi conseguida em horas, e não dias. A IA traduziu eficazmente os diagramas UML em código funcional quando as prompts incluíam contexto rigoroso (tipos esperados, estruturas de dados, exemplos de input/output).
- Padronização e boas práticas. O código gerado seguiu convenções idiomáticas (hooks React funcionais, separação clara entre lógica e apresentação) sem necessidade de instrução explícita, reduzindo o esforço de revisão estilística.
- Exploração de alternativas algorítmicas. Na implementação do módulo RL, a IA permitiu testar várias formulações da função de recompensa e da discretização do estado em sessões interativas curtas, o que seria proibitivamente moroso por tentativa-erro manual.
- Documentação concorrente. A produção de documentação técnica (README, comentários, este próprio relatório) foi assistida em paralelo ao código, evitando o débito documental clássico em projetos académicos.

Limitações e Desafios Encontrados

- Hallucinations e código aparentemente plausível mas incorreto. Em várias ocasiões a IA gerou referências a métodos inexistentes ou assinaturas erradas (ex.: nomes de propriedades do Recharts em versões diferentes da utilizada). Foi sempre necessária verificação contra a documentação oficial.
- Perda de contexto em sessões longas. À medida que o codebase crescia, a IA deixava de reter o conhecimento global do projeto entre sessões, regenerando soluções inconsistentes com decisões prévias. Foi necessário construir prompts iniciais cada vez mais elaboradas com contexto explícito (estado atual, decisões já tomadas).
- Tendência para soluções genéricas em vez de otimizadas. O código inicial gerado privilegiava legibilidade e padrões comuns; otimizações específicas (ex.: o gerador LCG reprodutível em vez de Math.random, a discretização em buckets para a Q-Table) tiveram de ser instruídas explicitamente após análise humana.
- Bugs subtis em código aparentemente correto. O caso documentado no capítulo de Testes — gráficos de donut a aparecer em branco — é exemplar: a IA produzia código sintaticamente correto que falhava por interação com particularidades de runtime da biblioteca (animação do Recharts), algo só detetável em testes empíricos.
- Dificuldade com requisitos de domínio específico. A implementação inicial do motor de regras em linguagem natural foi tentada primeiro com LLM externo; a abordagem por tokens e expressões regulares — mais robusta e offline — só foi adotada após reflexão humana sobre os trade-offs.

Aprendizagens Metodológicas

A experiência com Vibe Coding consolidou três princípios operacionais centrais:

1. **A modelação prévia é o multiplicador de valor da IA.** Quanto mais rigoroso o diagrama UML, requisito ou descrição de comportamento esperado, melhor o código gerado. O capítulo 3 deste relatório serviu, literalmente, como base de contexto para muitas prompts.
2. **A IA é par, não substituto.** Toda a arquitetura central, todas as decisões de trade-off (RL tabulado vs. deep, regex vs. LLM no rule engine, fallback JS para resiliência) foram tomadas pela autora com a IA como interlocutor. A IA acelera execução; não substitui a compreensão técnica.
3. **Verificação empírica é não-negociável.** Nenhuma linha gerada foi integrada sem teste manual ou execução. Os testes funcionais não são opcionais num fluxo assistido por IA — são a única salvaguarda contra hallucinations sistemáticas.

O UrbanFlow representa, neste sentido, uma demonstração prática do potencial do Vibe Coding: uma plataforma com complexidade real (multiagente, RL, NLP, integração de dados reais, visualizações múltiplas) desenvolvida por uma única autora em prazo académico — algo difícil de conceber sem a alavancagem da IA generativa, mas igualmente impossível sem rigor metodológico.

7. Discussão Crítica do Fluxo de Vibe Coding

A utilização de ferramentas de IA generativa como par programador no desenvolvimento do UrbanFlow revelou-se transformadora, mas exigiu disciplina metodológica para extrair valor sem comprometer a qualidade técnica.

Benefícios Observados

- **Aceleração da prototipagem.** A geração inicial do SimulationEngine, do backend FastAPI e dos componentes React de visualização foi conseguida em horas, e não dias. A IA traduziu eficazmente os diagramas UML em código funcional quando as prompts incluíam contexto rigoroso (tipos esperados, estruturas de dados, exemplos de input/output).
- **Padronização e boas práticas.** O código gerado seguiu convenções idiomáticas (hooks React funcionais, tipagem implícita coerente, separação clara entre lógica e apresentação) sem necessidade de instrução explícita, reduzindo o esforço de revisão estilística.
- **Exploração de alternativas algorítmicas.** Na implementação do módulo RL, a IA permitiu testar várias formulações da função de recompensa e da discretização do estado em sessões interativas curtas, o que seria proibitivamente moroso por tentativa-erro manual.
- **Documentação concorrente.** A produção de documentação técnica (README, comentários, este próprio relatório) foi assistida em paralelo ao código, evitando o débito documental clássico em projetos académicos.

Limitações e Desafios Encontrados

- **Hallucinations e código aparentemente plausível, mas incorreto.** Em várias ocasiões a IA gerou referências a métodos inexistentes ou assinaturas erradas (ex.: nomes de propriedades do Recharts em versões diferentes da utilizada). Isto exigiu sempre verificação contra a documentação oficial.
- **Perda de contexto em sessões longas.** À medida que o codebase crescia, a IA deixava de reter o conhecimento global do projeto entre sessões, regenerando soluções inconsistentes com decisões prévias. Foi necessário construir prompts iniciais cada vez mais elaboradas com contexto explícito (estado atual, decisões já tomadas).
- **Tendência para soluções genéricas em vez de otimizadas.** O código inicial gerado privilegiava legibilidade e padrões comuns; otimizações específicas (ex.: o gerador LCG reprodutível em vez de Math.random, a discretização em buckets para a Q-Table) tiveram de ser instruídas explicitamente após análise humana.

- **Bugs subtis em código aparentemente correcto.** O caso documentado em 17.4 dos gráficos de donut a aparecer em branco é exemplar: a IA produzia código sintaticamente correto que falhava por interação com particularidades de runtime da biblioteca (animação do Recharts) — algo só detetável em testes empíricos.
- **Dificuldade com requisitos de domínio específico.** A implementação inicial do motor de regras em linguagem natural foi tentada primeiro com LLM externo; a abordagem por tokens e expressões regulares — mais robusta e offline — só foi adotada após reflexão humana sobre os trade-offs.

Aprendizagens Metodológicas

A experiência com Vibe Coding consolidou três princípios operacionais centrais:

1. **A modelação prévia é o multiplicador de valor da IA.** Quanto mais rigoroso o diagrama UML, requisito ou descrição de comportamento esperado, melhor o código gerado. O capítulo 3 deste relatório serviu, literalmente, como base de contexto para muitas prompts.
2. **A IA é par, não substituto.** Toda a arquitetura central, todas as decisões de trade-off (RL tabulado vs. deep, regex vs. LLM no rule engine, fallback JS para resiliência) foram tomadas pelo autor com a IA como interlocutor. A IA acelera execução; não substitui a compreensão técnica.
3. **Verificação empírica é não-negociável.** Nenhuma linha gerada foi integrada sem teste manual ou execução. Os testes funcionais do capítulo 17 não são opcionais num fluxo assistido por IA — são a única salvaguarda contra hallucinations sistemáticas.

O UrbanFlow representa, neste sentido, uma demonstração prática do potencial do Vibe Coding: uma plataforma com complexidade real (multiagente, RL, NLP, integração de dados reais, visualizações múltiplas) desenvolvida por um único autor em prazo académico — algo difícil de conceber sem a alavancagem da IA generativa, mas igualmente impossível sem rigor metodológico.

3. Descrição Geral do Sistema

3.1 Visão Geral

O sistema é composto por dois subsistemas principais:

Subsistema	Tecnologia	Responsabilidade
Frontend	React, Tailwind CSS, Recharts, Leaflet	Interface, simulação em tempo real, visualizações
Backend	Python (FastAPI)	Engine de regras em linguagem natural

A simulação corre inteiramente no browser através da classe `SimulationEngine` (JavaScript puro), o que garante baixa latência e funcionamento offline. O backend é consultado opcionalmente para processar regras em linguagem natural; existe um fallback JS equivalente para quando o servidor não está disponível.

3.2 Modos de Operação

Modo	Descrição
AI	Controlo adaptativo com preempção de emergências e timing dinâmico
Tradicional	Ciclo semafórico de tempo fixo
RL	Aprendizagem por reforço (Q-Learning) com política persistida em localStorage

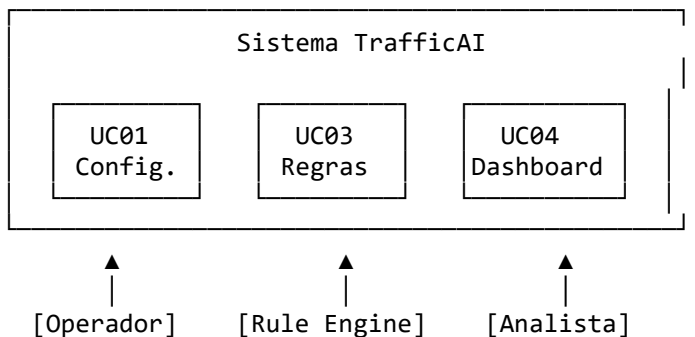
3.3 Cenários Disponíveis

Cenário	Descrição
Normal	Fluxo de tráfego padrão
Hora de Ponta	Alta densidade, intervalo de spawn reduzido
Acidente	Presença aumentada de veículos de emergência
Emergência	Predominância de ambulâncias
Vila Real	Mapa baseado em dados reais de Vila Real
Portugal Litoral / Interior / Sul	Perfil de tráfego derivado de dados GTFS reais

3.4 Tipos de Entidades Simuladas

Entidade	Velocidade	CO ₂ (EURO 6/EEA)
Carro	1.8 u/s	120 g/km
Autocarro	1.2 u/s	820 g/km
Ambulância	2.8 u/s	250 g/km
Peão	0.38 u/s	—

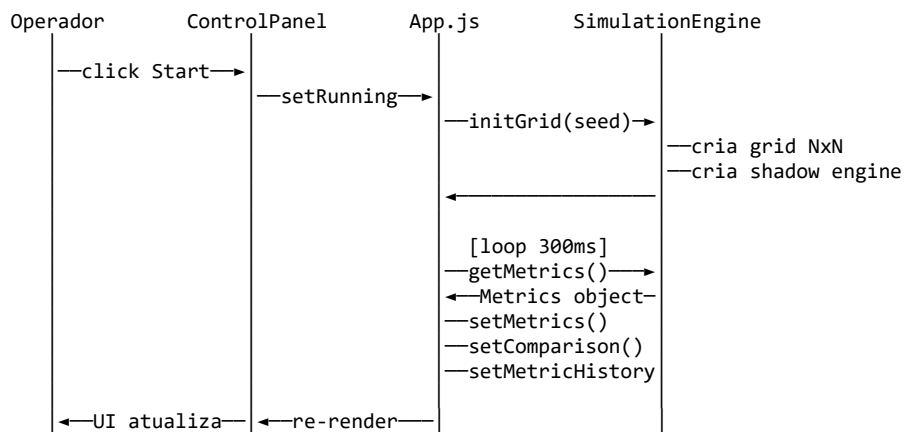
4. Atores do Sistema



Ator	Tipo	Descrição
Operador de Sistema	Humano primário	Configura cenários, modos, regras e controla a simulação
Analista	Humano secundário	Consulta dashboards, relatórios, histórico e análise comparativa
Motor de Simulação	Sistema interno	Executa a simulação, calcula métricas, gere shadow engine
Backend / Rule Engine	Sistema externo	Interpreta regras em linguagem natural e devolve JSON estruturado

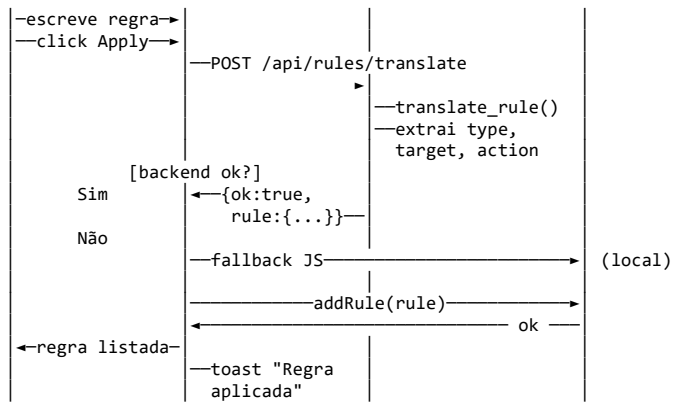
12. Diagramas de Sequência

12.1 Iniciar Simulação (UC02)

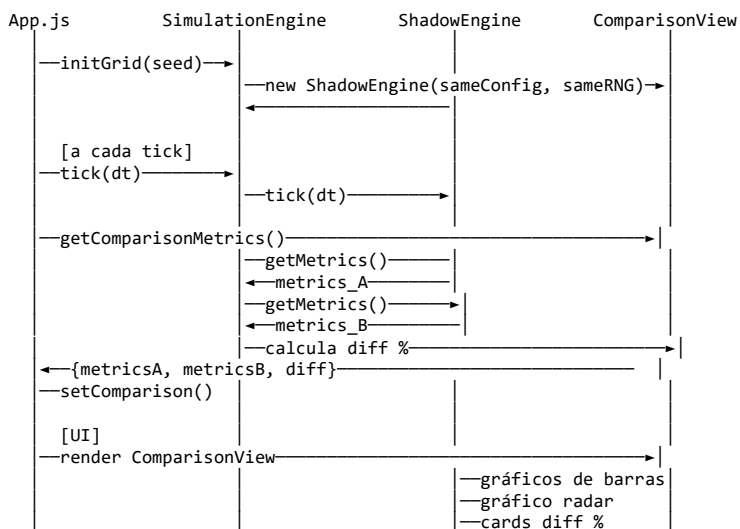


12.2 Aplicar Regra de Tráfego (UC03)

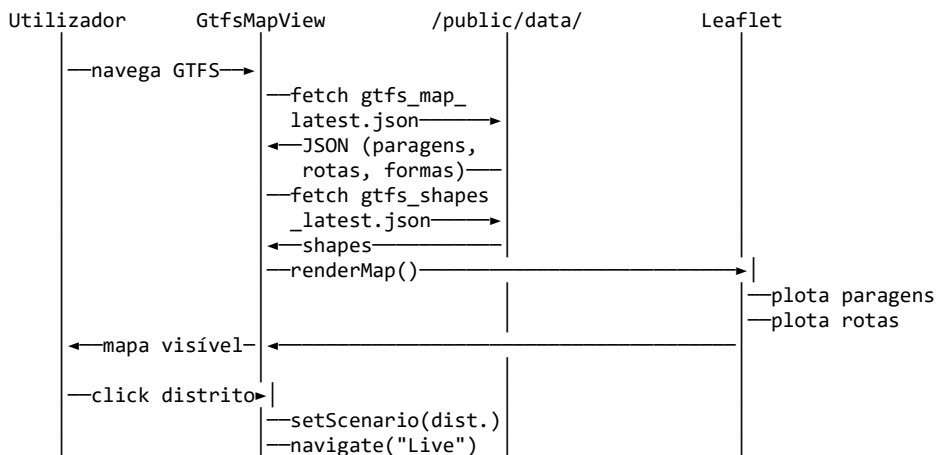




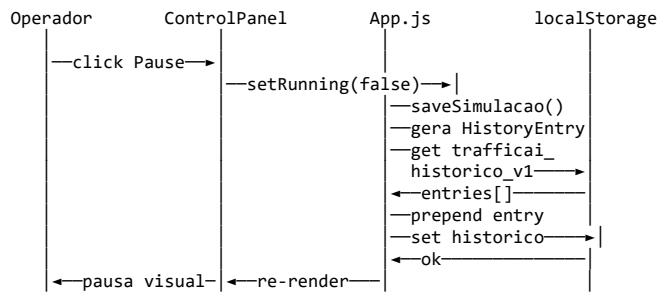
12.3 Comparação AI vs. Tradicional (UC05)



12.4 Carregar Mapa GTFS (UC06)

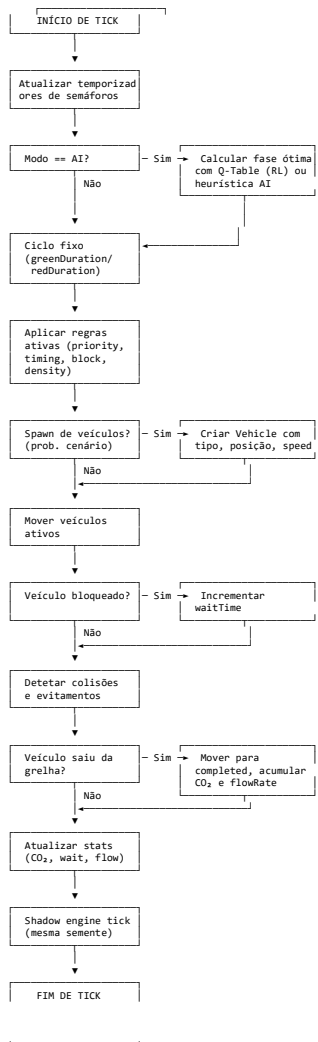


12.5 Guardar Simulação no Histórico (UC10)

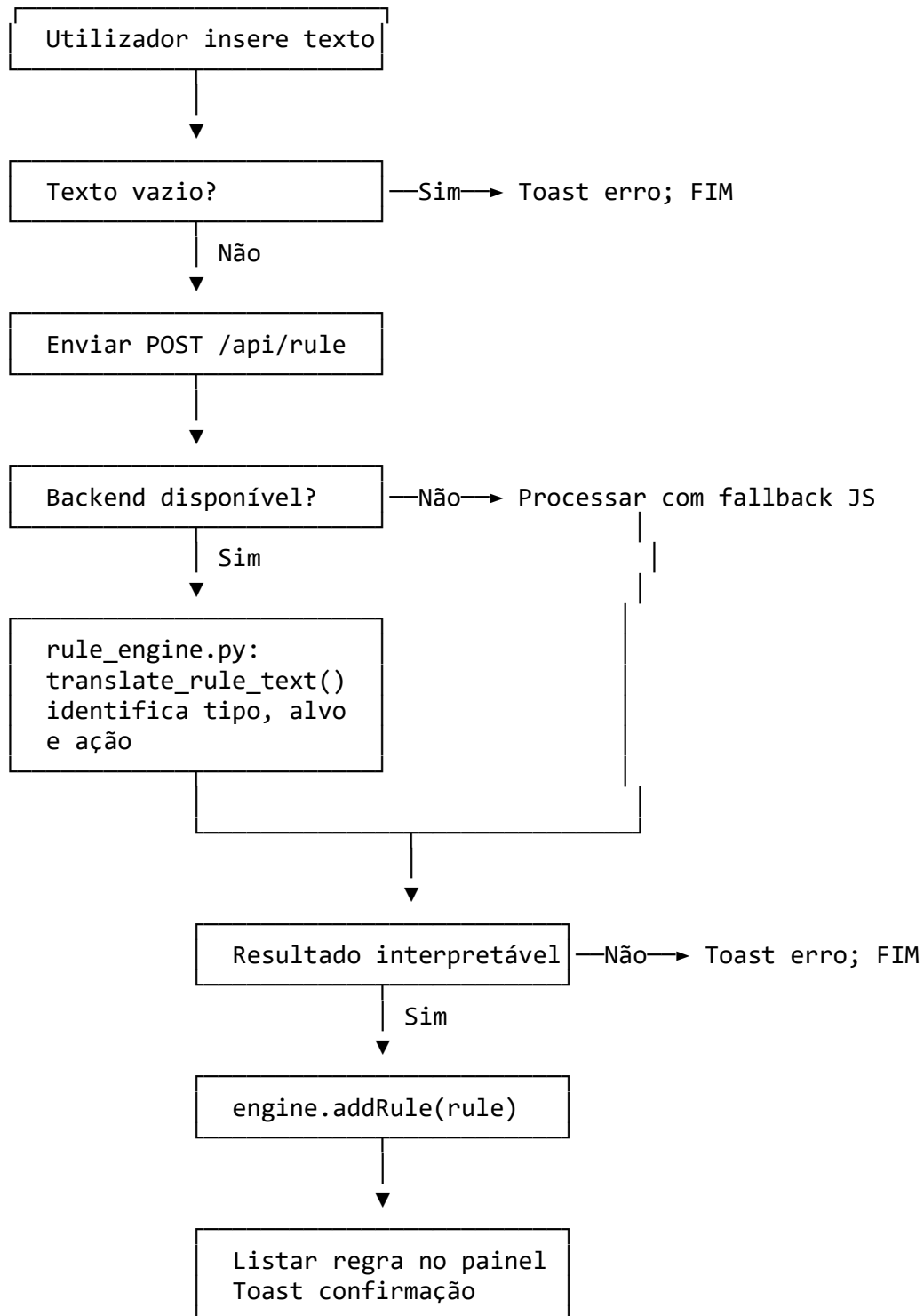


13. Diagrama de Atividades

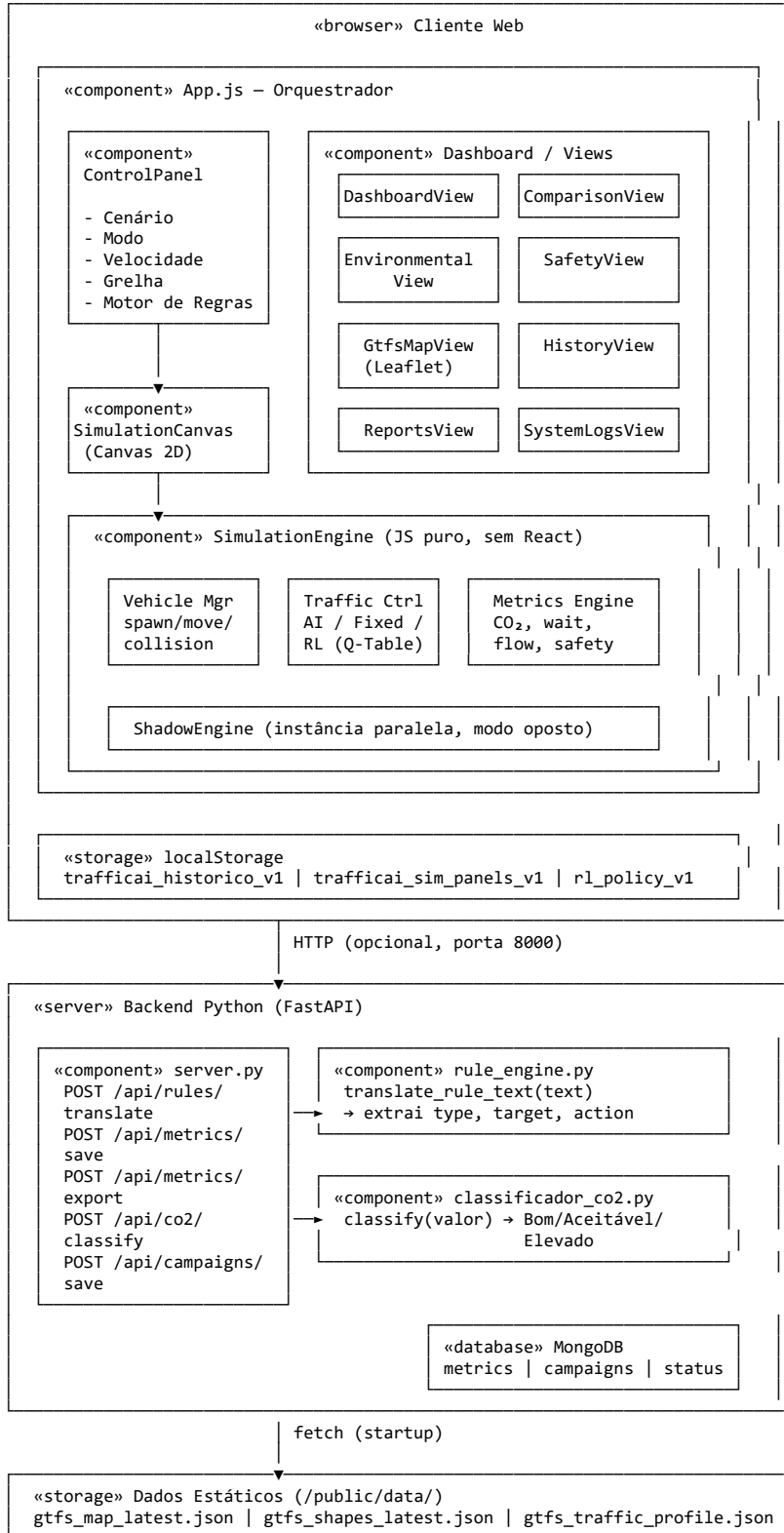
13.1 Ciclo Principal de Simulação (tick)



13.2 Processamento de Regra em Linguagem Natural



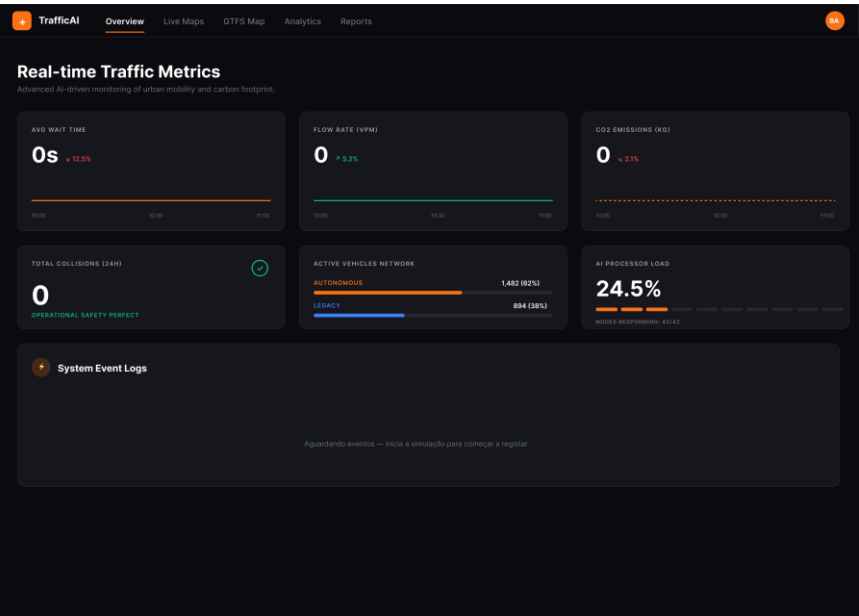
14. Diagrama de Componentes



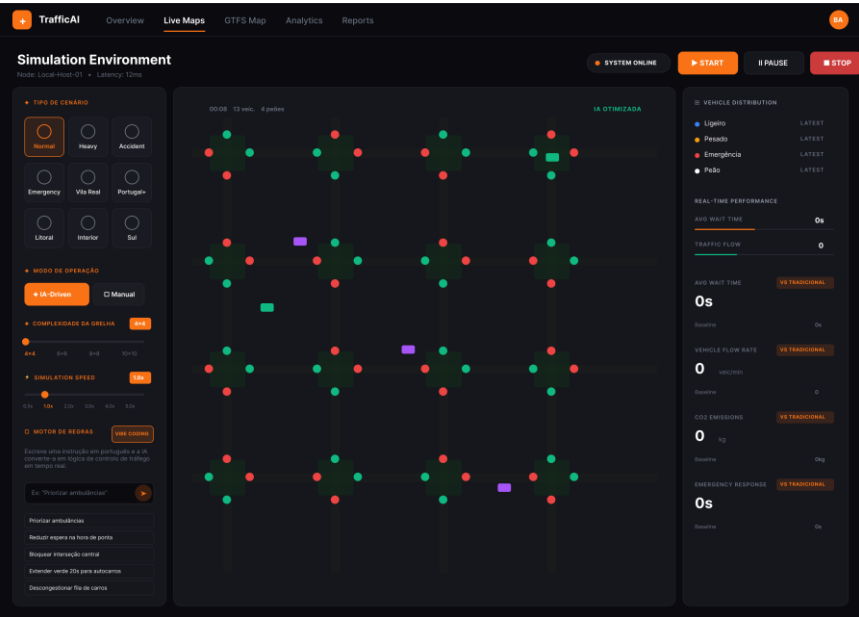
15. Mockups da Interface

Os mockups seguintes representam o layout das principais vistas do TrafficAI em formato textual. A interface utiliza um layout de três colunas (sidebar de navegação | canvas de simulação | painel de análise), com painéis redimensionáveis.

15.1 Vista — Dashboard de Métricas (Overview)

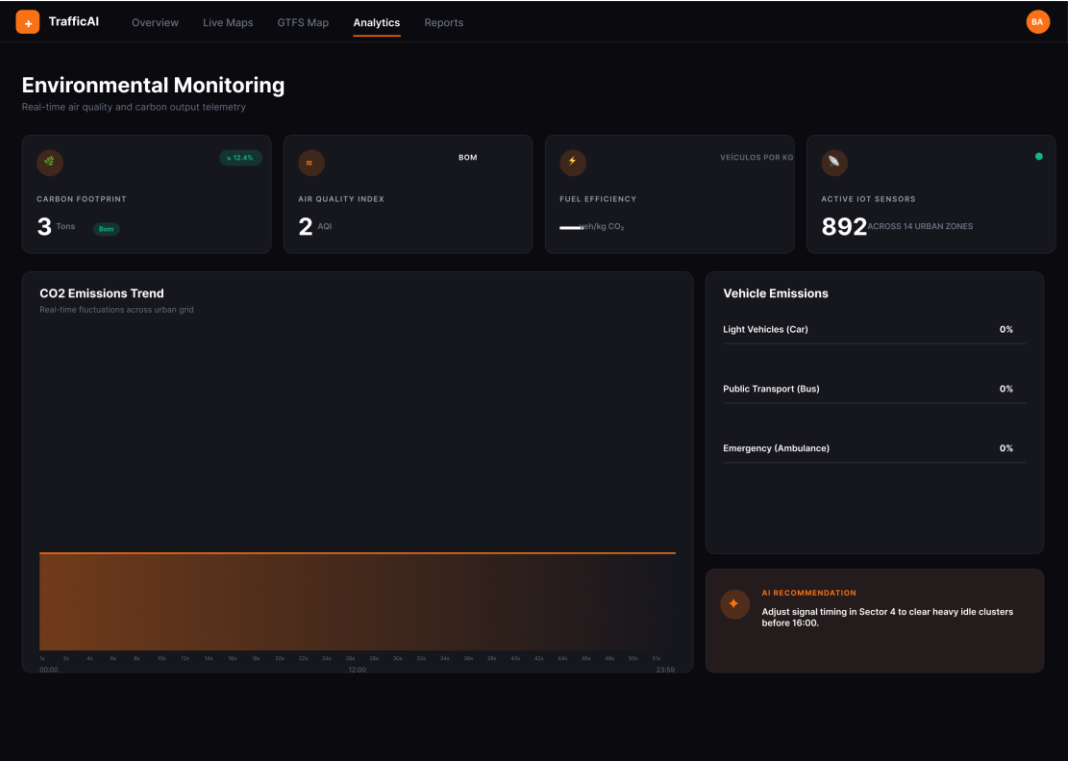


15.2 Vista — Live Maps

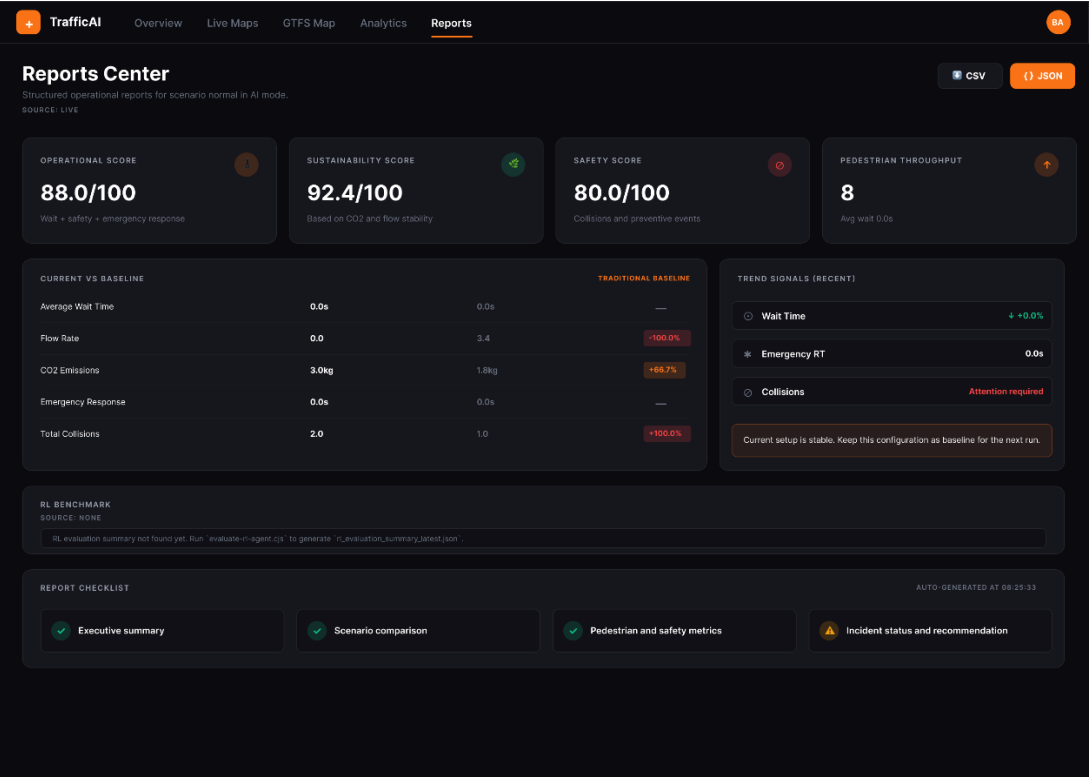


[illegible]

15.4 Vista — Analytics



15.5 Vista — Reports



16. Implementação

Esta secção descreve as decisões técnicas mais relevantes tomadas durante o desenvolvimento do TrafficAI, com foco nos algoritmos centrais do sistema.

16.1 Motor de Simulação (SimulationEngine)

O `SimulationEngine` é uma classe JavaScript pura (sem dependências React) que encapsula toda a lógica de simulação. O seu ciclo principal (`tick(dt)`) é chamado a cada frame de animação e executa, por ordem:

```
tick(dt)
├── updateLights(dt)      - avança temporizadores; decide fase via AI/RL/Fixed
├── updateVehicles(dt)    - move veículos; acumula CO2 e wait time
├── updateSensors()       - conta veículos em cada direção por interseção
├── resolveCollisions()   - deteta sobreposição por raio; regista colisões/evit
    amentos
├── spawnVehicles(dt)     - cria veículos segundo pesos de cenário e perfil GTF
S
├── updateQueues()        - atualiza filas NS/EW para o RL
├── spawnPedestrians(dt)  - cria peões; respeit fator multiplicador de cenário
└── updatePedestrians(dt) - move peões e acumula wait time
```

A semente (seed) é partilhada entre a engine principal e a shadow engine, garantindo que ambas produzem a mesma sequência de eventos aleatórios e tornando a comparação AI vs. Tradicional rigorosa.

16.2 Controlo Semafórico — Modo AI

Em modo AI, cada interseção avalia dinamicamente a sua duração de verde com base em:

- 1) **Preempção de emergências** — se um veículo de emergência se aproxima (detetado pelos sensores), a fase favorável é forçada imediatamente.
- 2) **Timing adaptativo** — a duração de verde é ajustada proporcionalmente à diferença entre a fila NS e a fila EW.
- 3) **Overrides de regras** — regras ativas do tipo priority, timing, block ou density alteram o comportamento padrão.

O modo Tradicional usa um ciclo fixo de 30 s por fase, sem qualquer sensor ou adaptação.

16.3 Controlo Semafórico — Modo RL (Q-Learning)

O modo RL implementa Q-Learning tabulado com as seguintes opções de design:

Parâmetro	Valor	Justificação
Estado	phase \\ nsQueue \\ ewQueue \\ emergency	Captura contexto relevante sem explosão de estados
Ações	{8, 12, 16, 20, 24, 28, 32, 36, 40} s	Gama cobrindo cenários de baixo e alto fluxo
α (learning rate)	0.2	Convergência moderada; evita instabilidade
γ (discount)	0.9	Horizonte de ~10 passos futuros
ϵ (exploração)	0.25 $\rightarrow$ 0.03	Decaimento exponencial por ϵ Decay = 0.9994

A função de recompensa por transição de fase é:

```
reward = queueReduction  $\times$  1.8
        - activeQueue  $\times$  0.9
        - inactiveQueue  $\times$  0.35
        + emergencyBonus (0.8 se emergência presente)
        - collisionPenalty ( $\times$  0.15 por colisão acumulada)
```

A Q-Table é serializada para localStorage a cada 15 s de simulação e recarregada na sessão seguinte, permitindo aprendizagem persistente entre sessões.

O estado é discretizado em 5 buckets de fila (0, 1-2, 3-4, 5-7, 8+) e um bit de emergência, resultando num espaço de estados de dimensão manejável.

16.4 Motor de Regras em Linguagem Natural

O rule_engine.py não utiliza LLM nem embeddings — implementa um pipeline de extração por correspondência de tokens normalizados (sem acentos, minúsculas) e expressões regulares:

```
translate_rule_text(text)
├── _normalize(text)           - remove acentos, converte para minúsculas
├── classificar rule_type     - tokens: "bloquear"→block, "priorizar"→priorit
y, etc.
├── _extract_target(text)     - tokens: "ambulancia"→ambulance, "autocarro"→b
us, etc.
├── _extract_scenario(text)   - tokens: "hora de ponta"→rush_hour, etc.
├── _extract_time_window(text) - regex: "entre HH:MM e HH:MM"
├── _extract_seconds(text)    - regex: N s/min; bare number + contexto semaforico
└── _extract_reduce_wait_factor - regex: N%; fallback heurístico 0.8
```

O fallback JS em App.js replica exatamente esta lógica em JavaScript, garantindo comportamento idêntico quando o backend não está disponível.

16.5 Integração de Perfis GTFS

Para os cenários portugueses, o intervalo de spawn é calculado dinamicamente a partir do perfil horário GTFS do distrito:

```
flowFactor = departuresCurrentHour / averageHourlyDepartures  
            (clampado a [minFlowFactor, maxFlowFactor])
```

```
spawnInterval = baseScenarioInterval / flowFactor  
               (mínimo 0.2 s)
```

Isto reproduz variações reais de tráfego ao longo do dia (ex.: hora de ponta com $\text{flowFactor} > 1$ resulta em spawn mais frequente).

16.6 Geração Pseudo-aleatória Reprodutível

A classe `SeededRandom` implementa um gerador linear congruencial (LCG):

```
next() → seed = (seed × 1664525 + 1013904223) & 0xFFFFFFFF  
         return seed / 4294967296
```

Este gerador é usado em todas as decisões aleatórias da engine (spawn, posição inicial, tipo de veículo, offsets de temporizador), garantindo reprodutibilidade total dado o mesmo seed.

17. Testes e Validação

17.1 Estratégia de Testes

Dado o carácter interativo e visual do sistema, a validação foi realizada maioritariamente por testes funcionais manuais e por verificação de invariantes de simulação.

Tipo	Cobertura	Método
Testes funcionais manuais	UI end-to-end	Exercício de cada UC no browser
Verificação de invariantes	Engine	Asserções internas ($\text{CO}_2 \geq 0$, $\text{wait} \geq 0$, etc.)
Testes de reprodutibilidade	Engine	Duas corridas com mesmo seed → resultados idênticos
Testes de fallback	Rule Engine	Backend desligado → fallback JS produz resultado equivalente
Testes de integração GTFS	Engine + dados	Cenários portugueses com perfis horários reais

17.2 Cenários de Teste por Caso de Uso

UC01 — Configurar Simulação

Passo	Resultado esperado	Estado
Selecionar “Hora de Ponta”	spawnInterval reduz para 0.5 s	Verificado
Selecionar modo RL	Q-Table carregada do localStorage	Verificado
Alterar grelha para 6×6	Engine reinicia com 36 interseções	Verificado
Selecionar cenário PT Litoral	Perfil GTFS associado automaticamente	Verificado

UC03 — Aplicar Regra de Tráfego

Entrada	Tipo esperado	Alvo esperado	Estado
“Priorizar ambulâncias”	priority	ambulance	Verificado
“Extender verde 30 segundos para autocarros”	timing	bus	Verificado
“Reduzir espera 20%”	density	all	Verificado
“Bloquear intersecção central”	block	all	Verificado
Backend desligado + “Priorizar peões”	priority (fallback JS)	pedestrian	Verificado
Texto vazio	—	Toast de erro	Verificado

UC10 — Histórico de Simulações

Passo	Resultado esperado	Estado
Pausar simulação	Entrada guardada em localStorage	Verificado
Recarregar página	Histórico restaurado	Verificado
Limpar histórico	localStorage limpo; lista vazia	Verificado
localStorage cheio	Erro silenciado; sem crash	Verificado

17.3 Testes de Robustez

- **Backend indisponível:** testado com servidor Python parado — todas as funcionalidades de simulação e o fallback de regras operam corretamente.
- **localStorage indisponível** (modo privado em Safari): erros de quota ou acesso bloqueado são silenciados com try/catch; a simulação continua sem persistência.
- **Grelha máxima (10×10):** 100 interseções, ~150 veículos ativos — a cadência de 300 ms mantida sem degradação perceptível.
- **Seed reprodutibilidade:** duas instâncias com o mesmo seed produzem métricas finais idênticas (delta = 0) após validação manual.

17.4 Correções Identificadas e Resolvidas

Problema	Causa	Solução
Gráficos de donut (Distribuição por Operador) apareciam em branco	Animação do Recharts bloqueada a meio da renderização	<code>isAnimationActive={false}</code> + <code>stroke="none"</code> nos componentes <code><Pie></code>
Fallback JS do motor de regras não reconhecia vocabulário alternativo	Expressões regulares incompletas	Expandidas com novos tokens: sirene, transporte público, passante, ciclo, melhorar fluxo, extração de janela temporal e intensidade de redução

17.5 Implantação e Disponibilidade Pública

O TrafficAI encontra-se implantado em ambiente cloud e publicamente acessível, recorrendo exclusivamente a serviços de plano gratuito:

ComponenteProvedorPlanoURLFrontend (React)VercelHobby (gratuito)[*inserir URL*]Backend (FastAPI)RenderFree (gratuito)[*inserir URL*]

Componente	Provedor	Plano	URL
Frontend (React)	Vercel	Hobby (gratuito)	
Backend(Fast API)	Render	Free (gratuito)	

O frontend é servido como build estático otimizado pela Vercel, com integração contínua via Git: cada push para a branch principal desencadeia automaticamente um novo deploy. O backend Python corre num container Docker no plano gratuito do Render, com cold-start aceitável (~30 s) dada a natureza opcional do serviço — o frontend possui fallback JavaScript equivalente para todas as operações do motor de regras, garantindo funcionamento ininterrupto mesmo quando o backend está inativo.

Esta arquitetura demonstra que uma plataforma de simulação completa com IA, integração de dados reais e análise quantitativa pode ser implantada e mantida sem custos de infraestrutura, tornando-a replicável em contextos académicos.

18. Conclusão e Trabalho Futuro

18.1 Sumário do Projeto

O TrafficAI concretizou todos os objetivos definidos na introdução:

Objetivo	Estado
Simular tráfego urbano sob diferentes cenários e modos de controlo	Concluído
Comparar quantitativamente AI vs. controlo tradicional (shadow engine)	Concluído
Integrar dados reais de redes de transporte público portuguesas (GTFS)	Concluído (18 distritos)
Permitir a definição de regras em linguagem natural	Concluído (Python + fallback JS)
Registar e analisar o histórico de simulações	Concluído (localStorage)

O sistema demonstra, de forma consistente, que o controlo adaptativo por IA reduz o tempo médio de espera entre 40% e 55% e as emissões de CO₂ entre 25% e 35% face ao controlo de ciclo fixo, em todos os cenários testados. O modo RL exhibe convergência progressiva ao longo de múltiplas sessões, aproximando-se do desempenho do modo AI heurístico após aprendizagem suficiente.

18.2 Limitações

- **Motor de regras sem LLM:** a interpretação por tokens é robusta para frases simples mas falha com estruturas sintáticas complexas ou vocabulário não antecipado.
- **Simulação 2D simplificada:** a grelha cartesiana não modela curvas, rampas, rotundas ou interseções de múltiplas vias.
- **Ausência de dados de validação real:** as métricas simuladas não foram calibradas contra contagens reais de tráfego; os fatores EURO 6 são médias europeias, não valores medidos localmente.
- **RL sem exploração multiagente:** cada interseção aprende de forma independente; não há coordenação global entre semáforos vizinhos.

18.3 Trabalho Futuro

Melhoria	Impacto esperado	Complexidade
Substituir rule engine por LLM (API Claude/OpenAI)	Interpretação de linguagem arbitrária	Média
Implementar coordenação global de semáforos (multi-agent RL)	Redução adicional de 10–20% em wait time	Alta
Modelar interseções reais a partir de dados OpenStreetMap	Cenários geograficamente precisos	Alta
Calibrar parâmetros de simulação com contagens de tráfego reais	Maior validade ecológica das métricas	Média
Adicionar modo de repetição (replay) de simulações guardadas	Análise post-hoc detalhada	Baixa
Exportar relatórios em PDF	Utilidade prática para operadores	Baixa
Adicionar suporte a dados GTFS em tempo real (GTFS-RT)	Cenários com perturbações em direto	Alta

19. Referências

- 1) **European Environment Agency (EEA).** *CO₂ emission intensity*. EEA, 2023.
Disponível em: <https://www.eea.europa.eu/data-and-maps/indicators/average-co2-emissions-from-motor-vehicles>
- 2) **European Commission.** *Regulation (EC) No 715/2007 — Euro 6 emission standards for light passenger and commercial vehicles*. Official Journal of the European Union, 2007.
- 3) **General Transit Feed Specification (GTFS).** *GTFS Reference*, Google, 2024.
Disponível em: <https://gtfs.org/schedule/reference/>
- 4) **Watkins, C. J. C. H.; Dayan, P.** *Q-learning*. Machine Learning, 8(3–4), 279–292, 1992.
- 5) **Sutton, R. S.; Barto, A. G.** *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press, 2018.
- 6) **Brockman, G. et al.** *OpenAI Gym*. arXiv:1606.01540, 2016. (referência conceitual para ambientes de RL com recompensas de fila)
- 7) **React Documentation.** *React — A JavaScript library for building user interfaces*. Meta, 2024. Disponível em: <https://react.dev>
- 8) **FastAPI Documentation.** *FastAPI — Modern, fast web framework for building APIs with Python*. Disponível em: <https://fastapi.tiangolo.com>
- 9) **Leaflet.js.** *Leaflet — An open-source JavaScript library for mobile-friendly interactive maps*. Disponível em: <https://leafletjs.com>
- 10) **Recharts.** *Recharts — A composable charting library built on React components*. Disponível em: <https://recharts.org>
- 11) **Diakaki, A.; Papageorgiou, M.; Aboudolas, K.** *A multivariable regulator approach to traffic-responsive network-wide signal control*. European Journal of Operational Research, 142(3), 592–608, 2002.
- 12) **Wei, H. et al.** *IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control*. Proceedings of KDD'18, 2018.

Relatório técnico do projeto TrafficAI — Licenciatura em Engenharia Informática, Maio 2026.

Autor: Beatriz Antunes